

BUSINESS REQUIREMENTS DOCUMENT

PortCo Monthly Reporting Dashboard *Web Application*

Document Version	1.0
Date	May 2026
Prepared By	Concertiv Product Team
Status	DRAFT

Table of Contents

Table of Contents.....	2
1. Executive Summary	4
2. Project Scope.....	4
2.1 In Scope	4
2.2 Out of Scope	4
3. Stakeholders & Roles.....	5
4. Functional Requirements	5
4.1 File Ingestion	5
4.2 Year Dropdown Filter.....	5
4.3 Cell Editability	6
4.4 Formula Engine	6
4.4.1 Aggregation Methods	6
4.4.2 Computed Metrics Registry	7
4.5 Report View Requirements	8
4.6 Number Formatting.....	8
5. Design Architecture.....	8
5.1 Layout & Navigation	8
5.2 Metric Table Column Layout.....	9
5.3 Colour System.....	9
5.4 Executive Summary Design.....	9
6. Code Architecture	10
6.1 Technology Stack.....	10
6.2 Project Structure.....	10
6.3 Core Data Model	11
6.4 Formula Registry Pattern.....	11
6.5 Computed Values via useMemo	12
6.6 File Parsing Flow	12
6.7 Period Aggregation Engine	13
7. Non-Functional Requirements.....	14
8. Assumptions & Constraints	14
8.1 Assumptions.....	14
8.2 Constraints	15
9. Testing Strategy.....	15

9.1 Unit Tests — Formula Engine	15
9.2 Component Tests	15
9.3 Integration Tests.....	15
9.4 UAT Acceptance Criteria	15
10. Delivery Milestones	16
11. Appendix.....	16
11.1 Editable Metric ID Reference	16
11.2 Revision History.....	17

1. Executive Summary

Concertiv currently manages PortCo monthly operational reporting via a multi-tab Excel workbook. This workbook tracks actuals across five departments (Product Development, Marketing, Sales, Customer Success, and Finance), compares them against budget targets, and produces period-over-period summaries. The process is manual, error-prone, and requires each analyst to understand hidden formula dependencies and colour-coded cell conventions.

This document defines requirements for replacing that workbook with a browser-based web application. The application will accept the existing Excel files as structured inputs, enforce all formula calculations automatically, render department-specific report views filtered by year, and display editable vs. computed cells with visual differentiation. No server infrastructure is required; the entire solution runs client-side in the browser.

Onboarding reporting has been explicitly descoped from this project as those rows are disabled (N/A) in the current workbook.

2. Project Scope

2.1 In Scope

- Upload and parse Actuals and Budget Excel files (.xlsx format)
- Year-based dropdown filter — show one year at a time (Jan–Dec)
- Five department output tabs: Product Development, Sales, Marketing, Customer Success, Finance
- Executive Summary tab with KPI cards per department
- Editable input cells with live formula recalculation
- Computed read-only cells derived from formula registry
- Actuals vs. Budget variance columns (delta and delta %)
- QTD, YTD, and LTM aggregations per metric
- Conditional formatting: green/red variance highlighting
- Sparkline trend indicator per metric row
- CSV export of current view

2.2 Out of Scope

- Onboarding department tab (cells are N/A in source workbook)
- Server-side storage, authentication, or user management
- Direct database integrations or API feeds
- Mobile/responsive layout below 1280px viewport width
- Multi-user real-time collaboration

3. Stakeholders & Roles

Role	Name / Team	Responsibility
Product Owner	Concertiv Product Team	Requirements sign-off, UAT approval
Finance Lead	Finance Department	Validate formula accuracy, budget data ownership
Department Leads	Each Dept. Head	Validate KPI definitions and metric accuracy per tab
Engineering Lead	Development Team	Technical architecture, implementation, deployment
QA Lead	QA Team	Test formula correctness, regression, UX validation

4. Functional Requirements

4.1 File Ingestion

The application must accept upload of the Concertiv workbook in .xlsx format. It parses two sheets: Actuals and Budget.

ID	Tab	Priority	Requirement
FR-01	Both	Must Have	Accept .xlsx file upload via drag-and-drop or file picker
FR-02	Both	Must Have	Parse Actuals sheet: columns A-F metadata, G+ monthly date columns
FR-03	Both	Must Have	Convert Excel serial date numbers to YYYY-MM string keys
FR-04	Budget	Must Have	Parse Budget sheet aggregation method column (Sum/Average/EoP) per row
FR-05	Both	Must Have	Skip all rows where Department = "Onboarding" or Department is blank
FR-06	Both	Should Have	Show parsed row count and date range summary after upload

4.2 Year Dropdown Filter

All output tabs must be filtered by a single Year dropdown in the application header. Data is never shown across multiple years simultaneously.

ID	Priority	Requirement	Notes
FR-10	Must Have	Year dropdown auto-populated from distinct years in loaded data	
FR-11	Must Have	Default selection: most recent year that contains any actual data	
FR-12	Must Have	Selected year drives all 12 monthly columns (Jan-Dec) across all tabs	Jan-Dec always shown
FR-13	Must Have	Months with no data display "—" not blank or zero	
FR-14	Must Have	QTD, YTD, LTM columns recalculate relative to selected year	

4.3 Cell Editability

The source Excel uses background colour to distinguish user-input cells from formula-derived cells. The web application must faithfully replicate this distinction with a visual coding system.

Cell Type	Excel Fill	Web App Behaviour
Editable (user input)	White / Yellow	White background, light border on hover, cursor changes to text. Click to enter inline edit mode (number input).
Computed (formula)	No fill / computed	Light gray background (#F5F5F5), read-only, "fx" badge on row label. Value recalculates on any upstream edit.
Computed Total	Black theme fill	Same as computed. Typically a Sum of sub-rows (e.g. Total Gross Growth).
Disabled / N/A	Dark fill (Onboarding)	Excluded entirely from the application. Row never rendered.

4.4 Formula Engine

All computed metrics must be calculated client-side in the browser using a centralised formula registry. Formulas must re-evaluate reactively when any upstream input changes.

4.4.1 Aggregation Methods

Every metric row carries an aggregation method that governs how monthly values are rolled up into period totals:

Method	Applies To	Calculation
Sum	ARR, Revenue, Costs, Counts	Sum of all non-null values in the period

Method	Applies To	Calculation
Average	Headcount, %, Rates, Cost per Squad	Mean of all non-null values in the period
EoP	ARR, Pipeline Value, Cash	Last non-null value in the period (End of Period)

4.4.2 Computed Metrics Registry

The following metrics are computed, never directly entered:

Metric	Department	Formula
% Completed	Prod Dev	Story Points Completed / Story Points Committed
Headcount (Total)	Prod Dev	Onshore + Offshore + Non-Squad
MQL to Opp Conversion %	Marketing	Mktg-sourced New Opps / MQLs
Expected Revenue of New Opp.	Marketing	Opp Value × Expected Win Rate
Mktg exp. per \$1 ARR	Marketing	L3M Avg Mktg Expense / L3M Avg New Logo ARR
New Pipeline Opportunities	Sales	Sales + Mktg + Referral Generated Opps.
Win Rate: Total	Sales	Won / (Won + Lost + Nurture)
EoP Value - Total	Sales	Early + Intermediate + Advanced Stage Value
Sales exp. per \$1 ARR	Sales	L3M Avg Sales Expense / L3M Avg New Logo ARR
Total Gross Growth	Cust. Success	Cross-sell + Upsell + Price
Total Net Growth	Cust. Success	Total Gross Growth + (-) Churn
Gross Growth %	Cust. Success	Total Gross Growth / Prior Month ARR
Churn %	Cust. Success	(-) Churn / Prior Month ARR
NRR %	Cust. Success	(Prior ARR + Net Growth) / Prior ARR
NPS Estimate	Cust. Success	(Very Satisfied - At-Risk) / Total Clients × 100
Adj. EBITDA	Finance	Revenue + all expense lines (expenses stored negative)
Gross Margin	Finance	(Revenue + CoS) / Revenue
NRR (annualized)	Finance	Product of last 12 monthly NRR % values
Net Debt	Finance	Debt & Debt-like Items + (-) Cash

4.5 Report View Requirements

ID	Priority	Requirement	Notes
FR-20	Must Have	12 monthly columns (Jan-Dec) shown for the selected year	No scrolling across years
FR-21	Must Have	Rows grouped by Category with sticky group header row	
FR-22	Must Have	QTD, YTD variance columns: Actual, Budget, Delta (\$), Delta (%)	
FR-23	Must Have	Green cell bg for favourable variance, red for unfavourable	Direction config per metric
FR-24	Must Have	Sparkline trend column (60px inline SVG) for last 12 months	
FR-25	Must Have	Computed rows display "fx" badge next to row label	
FR-26	Must Have	Clicking an editable cell activates inline number input	
FR-27	Should Have	CSV export button per tab downloads the current view	
FR-28	Should Have	Unsaved changes indicator on tab label (dot)	

4.6 Number Formatting

Units	Format	Example (positive)	Example (negative)
\$	\$#,##0	\$1,234,567	(\$1,234,567)
%	0.0%	12.3%	(12.3%)
#	0 (integers)	1,234	(1,234)
Missing	—	Shown in muted gray	Never blank, never NaN

5. Design Architecture

5.1 Layout & Navigation

The application follows a fixed-header single-page layout with persistent navigation.

Zone	Contents
Header bar	App logo/name (left) Upload buttons Year dropdown (right). Fixed, 64px tall, purple brand bar.
Tab nav	Executive Summary Prod Dev Sales Marketing Customer Success

Zone	Contents
	Finance. Sticky below header.
Main content	Scrollable metric table for the active tab. Full viewport width minus margins.
Tab toolbar	Export CSV button. Right-aligned, directly above the metric table.

5.2 Metric Table Column Layout

Each output tab renders the following column groups from left to right:

Group	Width	Columns	Notes	Sticky?
Row metadata	~220px	Label, Units		Yes
Monthly actuals	~72px each	Jan, Feb, ..., Dec (12 cols)	Selected year only	No
QTD	~80px each	Act Bgt Delta Delta%	4 cols, bordered group	No
YTD	~80px each	Act Bgt Delta Delta%	4 cols, bordered group	No
Trend	~60px	Sparkline SVG	Last 12 months	No

5.3 Colour System

Purpose	Hex	Usage
Brand primary	#512D6D	Header bar, active tab indicator, H1 heading colour
Editable cell bg	#FFFFFF	User-input cells, normal state
Computed cell bg	#F5F5F5	Formula-derived cells, group headers
Favourable variance	#E8F5E9	Delta cells where actual beats budget (good direction)
Unfavourable variance	#FFEBEE	Delta cells where actual misses budget (bad direction)
Missing data	#9E9E9E text	"—" displayed in muted gray on white background
Unsaved indicator	#FFC107 dot	Small amber dot on tab label when pending edits exist

5.4 Executive Summary Design

The Executive Summary tab renders one card per department. Each card follows this layout:

- Department name as card header (branded purple strip)
- 4–5 KPI chips in a 2-column grid inside the card
- Each KPI chip: metric label (small, muted) above; value (large, bold) below; variance badge (coloured, right-aligned)

- Click anywhere on the card navigates to the full department tab
- Cards laid out in a 2-column CSS Grid

6. Code Architecture

6.1 Technology Stack

Layer	Technology	Rationale
UI Framework	React 18 (hooks)	Component model, reactive state, minimal bundle size
Styling	Tailwind CSS	Utility-first, no runtime CSS-in-JS overhead
Excel Parsing	SheetJS (XLSX)	Industry-standard browser-side .xlsx parser; CDN deliverable
State Management	React useState/useMemo	Sufficient for single-user browser app; no Redux overhead
Build / Deploy	Vite + React	Fast HMR during development; optimised production bundle
Language	TypeScript	Type safety for formula registry and metric row model
Testing	Vitest + Testing Library	Unit tests for formula engine; component tests for render

6.2 Project Structure

```
src/
├── main.tsx                # App entry point
├── App.tsx                 # Root: header + tab router
├── types/
│   └── MetricRow.ts        # Core data model
├── state/
│   ├── useActuals.ts        # Editable actuals state
│   └── useBudget.ts         # Budget state (read-only after parse)
├── engine/
│   ├── formulas.ts          # Formula registry (all computed metrics)
│   ├── aggregations.ts      # Sum / Average / EoP helpers
│   ├── periodCalc.ts        # QTD, YTD, LTM window logic
│   └── formatter.ts         # Number display formatting
├── parsing/
│   ├── parseActuals.ts      # Actuals sheet parser
│   ├── parseBudget.ts       # Budget sheet parser
│   └── dateUtils.ts         # Excel serial → YYYY-MM
├── components/
│   ├── Header.tsx           # Fixed header + upload + year dropdown
│   ├── TabNav.tsx           # Department tab navigation
│   ├── MetricTable.tsx      # Core table renderer
│   ├── MetricRow.tsx        # Single row (editable or computed)
│   ├── CategoryHeader.tsx   # Sticky group header row
│   └── VarianceCell.tsx     # Coloured delta/delta% cell
```

```

├── Sparkline.tsx      # 60px inline SVG trend chart
├── KpiCard.tsx        # Executive summary card
├── UploadZone.tsx     # Drag-and-drop file upload
├── tabs/
│   ├── ExecSummary.tsx # Executive Summary tab
│   ├── ProdDev.tsx     # Product Development tab
│   ├── Sales.tsx       # Sales tab
│   ├── Marketing.tsx   # Marketing tab
│   ├── CustomerSuccess.tsx # Customer Success tab
│   └── Finance.tsx     # Finance tab
├── constants/
│   ├── editableRows.ts # Set of editable metric IDs
│   ├── computedRows.ts # Set of computed metric IDs
│   └── favorableDirection.ts # Per-metric variance direction config

```

6.3 Core Data Model

```

// types/MetricRow.ts

export type AggMethod = "Sum" | "Average" | "EoP";
export type Units     = "#" | "%" | "$";

export interface MetricRow {
  department: string; // "Product Development" | "Marketing" | ...
  category:   string; // "Squads" | "Production" | "Efficiency" | ...
  id:         string; // unique key, e.g. "Product Development # of Squads"
  lineItem:   string; // display label
  units:      Units;
  assignee:   string;
  isEditable: boolean; // true = user input; false = auto-computed
  aggMethod:  AggMethod;
  data: Record<string, number | null>; // key = "YYYY-MM"
}

// Application state shape
interface AppState {
  actuals: MetricRow[];
  budget:  MetricRow[];
  selectedYear: number;
  activeTab:   string;
  hasEdits:    boolean;
}

```

6.4 Formula Registry Pattern

```

// engine/formulas.ts

type MetricMap = Record<string, Record<string, number | null>>;
type FormulaFn = (data: MetricMap, month: string) => number | null;

export const FORMULAS: Record<string, FormulaFn> = {

  "Product Development % Completed": (d, m) =>
    safeDiv(
      d["Product Development Story Points Completed"]?.[m],
      d["Product Development Story Points Committed"]?.[m]
    ),

```

```

"Customer Success NRR %": (d, m) =>
  safeDiv(
    (prevMonthARR(d, m) ?? 0) + (d["Customer Success Total Net Growth"]?.[m] ?? 0),
    prevMonthARR(d, m)
  ),

"Finance Adj. EBITDA": (d, m) =>
  sum([
    d["Finance Revenue"]?.[m],
    d["Finance (-) Service/Delivery (CoS/CoGS)"]?.[m],
    d["Finance (-) Product"]?.[m],
    d["Finance (-) Technology"]?.[m],
    d["Finance (-) Sales"]?.[m],
    d["Finance (-) Marketing"]?.[m],
    d["Finance (-) Customer Success"]?.[m],
    d["Finance (-) G&A"]?.[m],
  ]),
  // ... all other formulas
};

// Helpers
function safeDiv(num?: number | null, den?: number | null, mult = 1) {
  if (den == null || den === 0) return null;
  return (num ?? 0) / den * mult;
}
function sum(vals: (number | null | undefined)[]) {
  const valid = vals.filter((v): v is number => v !== null && v !== undefined);
  return valid.length ? valid.reduce((a, b) => a + b, 0) : null;
}

```

6.5 Computed Values via useMemo

Computed metric values are never stored in state. They are derived on every render using `useMemo`, ensuring they stay in sync with editable inputs without any manual invalidation:

```

// Derives a MetricMap of { metricId: { "YYYY-MM": value } } for
// both editable actuals and all computed metrics.
const derivedActuals = useMemo(() => {
  // Step 1: build map from editable rows
  const baseMap: MetricMap = {};
  for (const row of actuals) {
    if (row.isEditable) baseMap[row.id] = row.data;
  }
  // Step 2: run formula registry over all months
  const allMonths = getMonthsForYear(selectedYear);
  for (const [metricId, fn] of Object.entries(FORMULAS)) {
    baseMap[metricId] = {};
    for (const month of allMonths) {
      baseMap[metricId][month] = fn(baseMap, month);
    }
  }
  return baseMap;
}, [actuals, selectedYear]);

```

6.6 File Parsing Flow

```

// parsing/parseActuals.ts
import * as XLSX from "xlsx";

```

```

export function parseActuals(file: File): Promise<MetricRow[]> {
  return new Promise(resolve => {
    const reader = new FileReader();
    reader.onload = (e) => {
      const wb = XLSX.read(e.target!.result, { type: "binary" });
      const ws = wb.Sheets["Actuals"];
      const raw = XLSX.utils.sheet_to_json(ws, { header: 1, defval: null });

      // Row 3 (index 2) contains the header; cols 0-6 = metadata, 7+ = months
      const headerRow = raw[2] as any[];
      const monthKeys = headerRow.slice(7).map(excelSerialToYYYYMM);

      const rows: MetricRow[] = [];
      for (const row of (raw as any[][]).slice(3)) {
        const dept = row[1];
        if (!dept || dept === "Onboarding") continue;
        const id = row[3] as string;
        rows.push({
          department: dept,
          category: row[2],
          id,
          lineItem: row[4],
          units: row[5],
          assignee: row[6] ?? "",
          isEditable: EDITABLE_IDS.has(id),
          aggMethod: AGG_METHOD[id] ?? "Sum",
          data: Object.fromEntries(
            monthKeys.map((k, i) => [k, row[7 + i] ?? null])
          )
        });
      }
      resolve(rows);
    };
    reader.readAsBinaryString(file);
  });
}

function excelSerialToYYYYMM(serial: number): string {
  const d = new Date(Math.round((serial - 25569) * 86400 * 1000));
  const y = d.getUTCFullYear();
  const m = String(d.getUTCMonth() + 1).padStart(2, "0");
  return `${y}-${m}`;
}

```

6.7 Period Aggregation Engine

```

// engine/periodCalc.ts

export function getQTD(data: Record<string,number|null>, year: number,
  lastMonth: number, method: AggMethod) {
  const qStart = Math.floor((lastMonth - 1) / 3) * 3 + 1;
  const months = range(qStart, lastMonth).map(m =>
    `${year}-${String(m).padStart(2, "0")}`);
  return aggregate(months.map(m => data[m] ?? null), method);
}

export function getYTD(data: Record<string,number|null>, year: number,
  lastMonth: number, method: AggMethod) {
  const months = range(1, lastMonth).map(m =>
    `${year}-${String(m).padStart(2, "0")}`);

```

```

    return aggregate(months.map(m => data[m] ?? null), method);
}

export function getLTM(data: Record<string,number|null>, currentMonthKey: string,
    method: AggMethod) {
    const months = Array.from({length: 12}, (_, i) => monthOffset(currentMonthKey, -i));
    return aggregate(months.map(m => data[m] ?? null), method);
}

export function aggregate(values: (number|null)[], method: AggMethod) {
    const valid = values.filter((v): v is number => v !== null);
    if (!valid.length) return null;
    if (method === "Sum") return valid.reduce((a, b) => a + b, 0);
    if (method === "Average") return valid.reduce((a, b) => a + b, 0) / valid.length;
    if (method === "EoP") return valid[valid.length - 1];
    return null;
}

```

7. Non-Functional Requirements

ID	Category	Priority	Requirement
NFR-01	Performance	Must Have	File parse + full render in < 2 seconds for the current workbook size (~130 rows, 45 months)
NFR-02	Performance	Must Have	Formula re-evaluation after a single cell edit in < 100ms
NFR-03	Compatibility	Must Have	Chrome 110+, Edge 110+, Firefox 110+, Safari 16+
NFR-04	Viewport	Must Have	Minimum supported width: 1280px. Layout degrades gracefully at 1024px.
NFR-05	Accessibility	Should Have	Keyboard navigable table cells; ARIA labels on all inputs and computed badges
NFR-06	Reliability	Must Have	Division by zero or null upstream data always renders "—" without crashing
NFR-07	Security	Must Have	No data leaves the browser. No API calls, no telemetry, no third-party tracking
NFR-08	Maintainability	Should Have	Formula registry fully covered by unit tests. New formulas addable by editing one file.

8. Assumptions & Constraints

8.1 Assumptions

- The source workbook structure (column layout, sheet names, row order) remains consistent between uploads.

- Department names in the Excel file exactly match the strings used in the editability classification table.
- Budget data is treated as read-only after upload; users edit only Actuals data in the app.
- All date columns in the Excel file are valid month-end Excel serial dates.
- The application will be used on corporate laptops with modern browsers; no IE11 support required.

8.2 Constraints

- No backend infrastructure: the entire solution runs in the browser.
- The Excel file structure must be maintained for parsing to work correctly.
- Onboarding tab is permanently excluded from scope.
- Large files (1,000+ rows, 100+ months) are not a target use case and performance targets are not set for that scale.

9. Testing Strategy

9.1 Unit Tests — Formula Engine

All formulas in the registry must have unit tests covering:

- Normal case: valid upstream inputs produce correct output
- Null upstream: any null input returns null (not NaN or 0)
- Division by zero: denominator = 0 returns null
- Boundary: single-month periods, LTM crossing year boundary

9.2 Component Tests

- MetricRow renders correct value, format, and background for editable vs computed
- VarianceCell applies green/red class based on favorable direction config
- Year dropdown filters table to 12 correct monthly columns
- Inline edit: click activates input, Enter commits, Escape reverts

9.3 Integration Tests

- Upload the actual workbook file → verify parsed row count equals expected
- Edit a Gross Growth input → verify NRR %, Adj. EBITDA, and Gross Margin all update
- Change year dropdown → verify all tab column headers change to selected year

9.4 UAT Acceptance Criteria

ID	Scenario	Pass Criteria
----	----------	---------------

ID	Scenario	Pass Criteria
UAT-01	Upload the Concertiv workbook	All 5 departments load with correct row counts and month columns
UAT-02	Switch Year dropdown to 2024	All tabs show Jan 2024 – Dec 2024 columns only
UAT-03	Edit Story Points Committed in Prod Dev	% Completed row updates immediately without page reload
UAT-04	Edit (-) Churn in Customer Success	NRR %, Total Net Growth, and Finance Adj. EBITDA all update
UAT-05	Verify QTD/YTD columns in Finance tab	Values match manual sum of monthly actuals for the period
UAT-06	Export CSV from Sales tab	Downloaded CSV contains all visible rows and columns for selected year

10. Delivery Milestones

Phase	Milestone	Deliverables	Duration
1	Foundation	Project scaffold, TypeScript types, file parser, formula registry with tests	Week 1–2
2	Core Table UI	MetricTable, MetricRow, CategoryHeader, VarianceCell, Sparkline components	Week 3–4
3	Department Tabs	All 5 department tabs with year filter and QTD/YTD columns	Week 5–6
4	Exec Summary	Executive Summary KPI cards, tab routing, inline edit flow	Week 7
5	QA & UAT	Formula accuracy validation, UAT with Finance team, bug fixes	Week 8–9
6	Release	Production build, deployment, handover documentation	Week 10

11. Appendix

11.1 Editable Metric ID Reference

The following table lists all metric IDs that are classified as EDITABLE (user-input) in the Actuals tab. All other metrics in those departments are computed by formula.

Department	Editable Metric IDs
------------	---------------------

Department	Editable Metric IDs
Prod Dev	# of Squads, # of Sprints, Story Points Committed, Story Points Completed, Product (%), Maintenance/Internal (%), Headcount - Squad - Onshore/Offshore/Non-Squad, Avg. Personnel Cost per Squad, P&T Overhead
Marketing	Marketing Qualified Leads, = Mktg sourced New Opp. Value, (x) Expected Win Rate, Mktg sourced pipeline EoP, Mktg sourced Won ARR, All Lead counts by channel, All Spend by channel, Headcount
Sales	New Logo ARR (\$), Sales/Marketing/Referral Generated Opps., Won, Lost+Nurture, Avg ACV, Avg Age of Opps Won, EoP Value/Count (Early/Intermediate/Advanced), Pipeline Weeks (Early/Intermediate/Advanced), Lost+Nurtured by stage, Headcount
Cust. Success	Cross-sell, Upsell, Price, (-) Churn, # of Clients, Reference Clients, Churned Clients, # Growth Clients, # Renewed Clients, Obtainment, NPS inputs (Very Satisfied/Satisfied/Dissatisfied/At-Risk), Headcount
Finance	ARR, Revenue, (-) Service/Delivery (CoS), (-) Product, (-) Technology, (-) Sales, (-) Marketing, (-) Customer Success, (-) G&A, Other, Debt & Debt-like Items, (-) Cash, Employees, VeArc, 3rd Party Providers

11.2 Revision History

Version	Date	Author	Changes
1.0	May 2026	Product Team	Initial draft — full requirements, design, and code architecture